git cheat sheet

Hebrew University · Bioinformatics · DNA Dungeon Edition

v2025

CONCEPTS & SETUP

THE THREE ZONES

```
Working Dir  →  Staging Area  →  Repository
  (you edit)      (git add)       (git commit)

  git add file     → moves to staging
  git commit       → seals into repo
  git restore file → discard working changes
  git restore      → unstage back to working
    --staged file
```

FIRST-TIME SETUP

`git config --global user.name "Name"`	your name
`git config --global user.email "e@mail"`	your email
`git config --global init.defaultBranch main`	default branch
`git config --global core.editor "code --wait"`	VS Code editor
`git config --global core.editor "nano"`	nano (Linux)
`git config --list`	show all config
`git init`	init repo here
`git status`	see current state

EDITOR CHEATSHEET

nano (Linux default)
type your message on line 1
Ctrl+X → Y → Enter to save & exit
Ctrl+C to cancel without saving

vim (may open unexpectedly)
press i to enter insert mode
type your message
press Esc then type :wq then Enter
to abort: Esc then :q! then Enter

VS Code (if core.editor = code)
type message in the file that opens
Ctrl+S to save, then close the tab

STAGING, COMMITTING & HISTORY

GIT ADD

`git add file.py`	stage one file
`git add src/`	stage a folder
`git add *.py`	stage by pattern
`git add .`	stage everything
`git add -p`	stage chunks interactively

GIT COMMIT

`git commit -m "msg"`	commit with message
`git commit`	opens editor for message
`git commit -am "msg"`	add tracked + commit
`git commit --amend -m "msg"`	fix last message
`git commit --amend --no-edit`	add to last commit

GIT LOG

`git log`	full history
`git log --oneline`	compact view
`git log --oneline --graph --all`	visual branch graph
`git log -n 5`	last 5 commits
`git log --author="Name"`	filter by author
`git show abc123`	inspect a commit

GIT RM

`git rm file.py`	remove from git + disk
`git rm --cached file.py`	untrack, keep locally
`git rm --cached -r data/`	untrack whole folder

Use `--cached` when you forgot to .gitignore something before committing it

DIFF & .GITIGNORE

GIT DIFF

`git diff`	unstaged changes
`git diff --staged`	staged vs last commit
`git diff HEAD`	all changes since last commit
`git diff abc123 def456`	between two commits
`git diff main..feature`	between two branches
`git diff --stat`	summary only
`git diff --word-diff`	word-level changes
`git difftool`	open in VS Code / tool

.GITIGNORE SYNTAX

*.pyc ignore by extension
data/ ignore whole folder
data/* ignore folder contents
!data/cfg.json exception — track this
**/*.tmp ignore everywhere
secrets.txt ignore specific file

`git check-ignore -v file`	why is this ignored?
`git rm --cached file`	untrack already-committed

DIFF TOOL SETUP

Set VS Code as diff tool (run once):

```
git config --global diff.tool vscode
git config --global difftool.vscode.cmd 'code --wait --diff $LOCAL $REMOTE'
```

`git difftool`	open diff in VS Code

Or use GitLens Source Control panel → click any changed file for visual side-by-side diff

UNDOING THINGS

GIT RESTORE (files only)

`git restore file.py`	discard working changes
`git restore .`	discard ALL unstaged
`git restore --staged file.py`	unstage a file
`git restore --staged .`	unstage everything
`git restore -SW file.py`	staged + working dir
`git restore --source HEAD~2 f.py`	file from 2 commits ago

History stays intact. Safe after push. Default source = HEAD.

GIT RESET (moves branch)

`git reset --soft HEAD~1`	undo commit, keep staged
`git reset --mixed HEAD~1`	undo commit, keep unstaged
`git reset --hard HEAD~1`	undo commit, delete changes
`git reset --hard HEAD~3`	go back 3 commits
`git reset --mixed abc123`	go back to specific commit

--hard after push = dangerous. Use revert instead.

GIT REVERT (safe after push)

`git revert abc123`	new commit undoing abc123
`git revert HEAD`	undo last commit safely
`git revert HEAD~3..HEAD`	revert range of commits
`git revert --no-commit abc123`	revert without committing

reset = rewrite history ⚠ after push
revert = add undo commit ✓ always safe

GIT STASH

`git stash`	hide all changes
`git stash push -m "desc"`	stash with name
`git stash list`	see all stashes
`git stash pop`	restore + delete from stash
`git stash apply stash@{2}`	apply specific stash
`git stash drop stash@{0}`	delete one stash
`git stash clear`	delete all stashes

git cheat sheet

Branches · Remote · Oops Scenarios · Advanced

v2025

BRANCHES

GIT BRANCH & SWITCH

`git branch`	list local branches
`git branch -a`	list all incl. remote
`git branch new-branch`	create branch
`git switch -c new-branch`	create and switch ✓ preferred
`git switch -c branch abc123`	create at specific commit
`git switch branch`	switch to branch
`git switch -`	go back to previous branch
`git branch -d branch`	delete (safe, if merged)
`git branch -D branch`	force delete
`git branch -m old new`	rename branch

GIT MERGE

`git merge feature`	merge feature into current
`git merge --no-ff feature`	always create merge commit
`git merge --squash feature`	squash all into one commit
`git merge --abort`	cancel merge in progress
`git cherry-pick abc123`	apply one commit from elsewhere

Fast-forward: main slides forward, no new commit
Merge commit: two histories joined → commit with 2 parents
Conflict: edit file → remove markers → git add → commit

NAVIGATING HISTORY

`git switch --detach abc123`	visit commit (tourist)
`git switch --detach HEAD~3`	3 commits back
`git switch main`	return to present
`git switch -c branch`	save detached commits → new branch
`git reflog`	every position HEAD has been
`git reset --hard HEAD@{3}`	go back 3 HEAD moves

Before traveling → **stash or commit**. Reflog keeps ~90 days. **Almost nothing is truly lost.**

REMOTE & GITHUB

REMOTE SETUP

`git remote -v`	show remotes
`git remote add origin <url>`	add remote
`git remote remove origin`	remove remote
`git clone <url>`	clone repo locally
`git clone <url> folder`	clone into folder

GIT PUSH

`git push -u origin main`	first push, set upstream
`git push`	push current branch
`git push origin feature`	push specific branch
`git push --force-with-lease`	safe force push
`git push origin --delete branch`	delete remote branch

GIT FETCH & PULL

`git fetch`	download, don't merge
`git fetch --prune`	fetch + clean deleted branches
`git pull`	fetch + merge
`git pull --rebase`	fetch + rebase (cleaner)
`git diff main origin/main`	compare local vs remote

fetch = safe, just looks | **pull** = fetch + merges

GIT REBASE

`git rebase main`	replay commits onto main
`git rebase --abort`	cancel rebase
`git rebase --continue`	continue after conflict
`git rebase -i HEAD~5`	interactive: clean up 5 commits

Options: **pick reword fixup squash drop**
Never rebase pushed shared branches

OOPS SCENARIOS

COMMON FIXES

SITUATION	FIX
Typo in last commit message	`git commit --amend -m "new msg"`
Forgot file in last commit	`git add f && git commit --amend --no-edit`
Staged wrong file	`git restore --staged file.py`
Broke a file, want original back	`git restore file.py`
Committed file that should be ignored	`git rm --cached file → fix .gitignore`
Blanket ignore catching needed file	`git check-ignore -v file → add ! exception`
Need to undo pushed commit safely	`git revert abc123`
Merge conflict	`edit file → remove markers → git add → commit`

BRANCH & REMOTE FIXES

SITUATION	FIX
Committed on wrong branch (not pushed)	`git switch -c right → reset --hard HEAD~1 on wrong`
Committed on wrong branch (pushed)	`same + git push --force-with-lease`
Local & remote diverged	`git pull --rebase OR git push --force-with-lease`
Deleted branch by accident	`git reflog → git switch -c branch abc123`
Reset --hard regret	`git reflog → git reset --hard HEAD@{n}`
Detached HEAD with commits made	`git switch -c new-branch (before leaving!)`
Messy commit history	`git rebase -i HEAD~n`
Want one commit from another branch	`git cherry-pick abc123`

RESTORE VS RESET VS CHECKOUT VS STASH · DIRECTED GRAPH

RESTORE

Fix **files**
History **intact**
Safe after push ✓
Use --source for old commits

RESET

Moves **branch pointer**
Rewrites history
⚠ after push
--soft/--mixed/--hard

CHECKOUT (OLD)

Old swiss-army knife
Use **switch** for branches
Use **restore** for files
Still works everywhere

STASH

Temporarily **shelve** work
Purely local ✓
Stack-based (LIFO)
No HEAD~n support

DAG

```
HEAD→branch→commit
commits point backwards
branch = 40-char hash file
cat .git/HEAD
cat .git/refs/heads/main
reflog = 90 day diary
```

MY NOTES